本形學
Ontypology

- 從異同到分合，從分合到形，從形到理解這條路，大腦走了億萬年。
- 邏輯學百年前是哲學，當看清楚後是顯學(科學)
- 今探討「形」(大腦概念) 是哲學，本形學將「形」，看清楚轉成顯學(科學)，並實践化 Mind AI 。

本形学 Ontypology ★

异同 → 分合 → 形 → 理解 → 显学(科学)



IC設計大腦：準確生成 Verilog 的 Type-Driven 指南

從混沌到可驗證的真理 —— 工程師的 AI 準確使用手冊

[核心法則：設計 = 形（Type）+ 數]



寫在前面：不可妥協的三約定



約定一・形 = Type (絕非 Form)

「形」的英文一律是 Type。形不是目錄或分類標籤；形是一口立得出判準 ∩(T)、撐得起推理的嚴密結構。



約定二・理論只用不教

空間本形的三括號、交集判準 ∩ (T) 與 remove 三三級，本書直接作為實戰工具套用，不再贅述底層本形學論證。

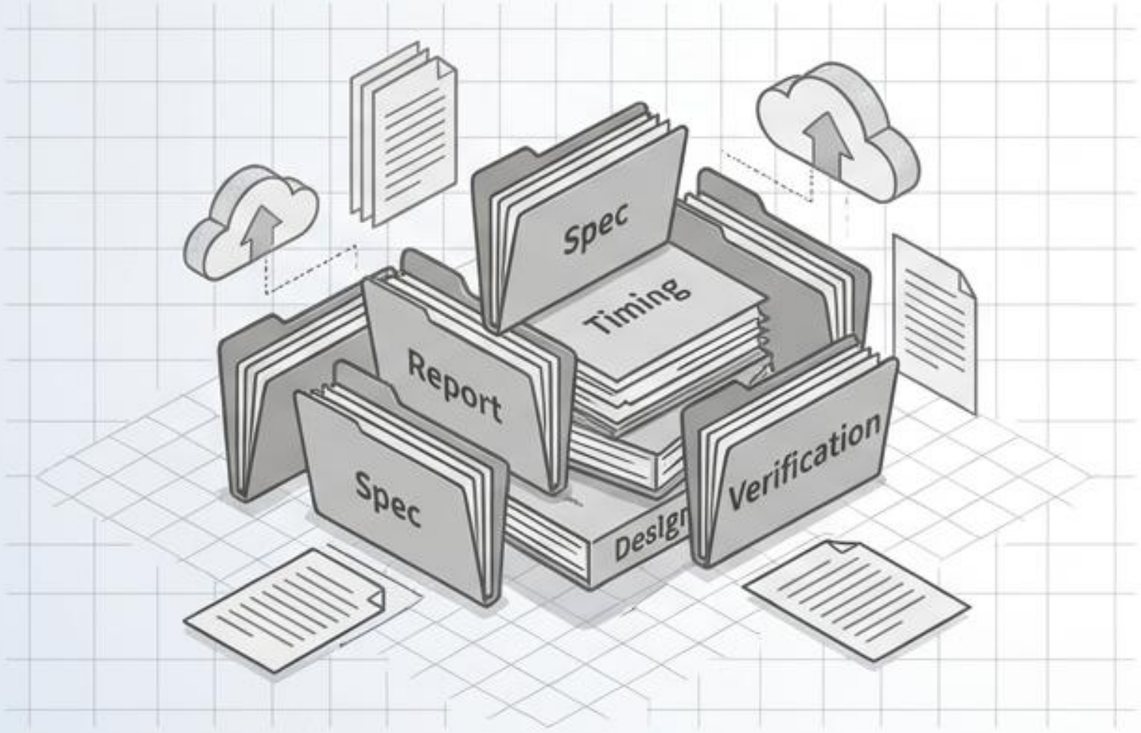


約定三・平台只引不述

Mind AI 僅作為承載大腦的「底層平台」。本書專注於在此平台之上，工程師「怎麼把 IC 設計做準」，而非探討平台本身的開發。

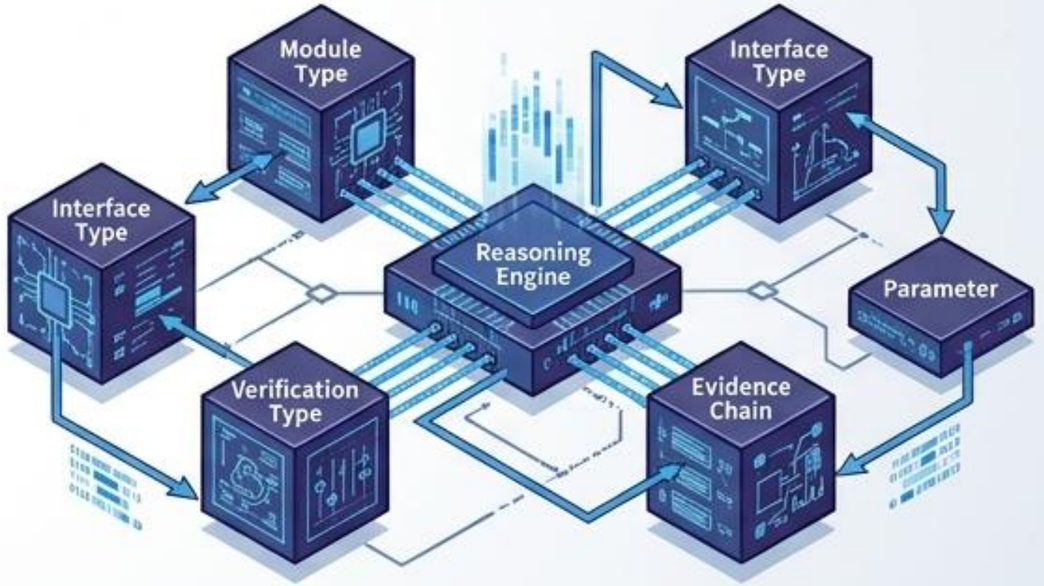
現狀：散裝的死文件庫

- 規格書躺在雲端，時序圖無版本控制。
- 驗證報告與設計脫節。
- 文件找得到，答案問不出。



大腦 = 可推理的設計副本

- **知識**：形倉庫可推理
- **思維**：招式盤可調用
- **演化**：證據鏈可累積
- **專案情境**：改了 8N1 規格，大腦自動列出受影響的介面形、模組形與驗證形，而非等驗證掛掉。

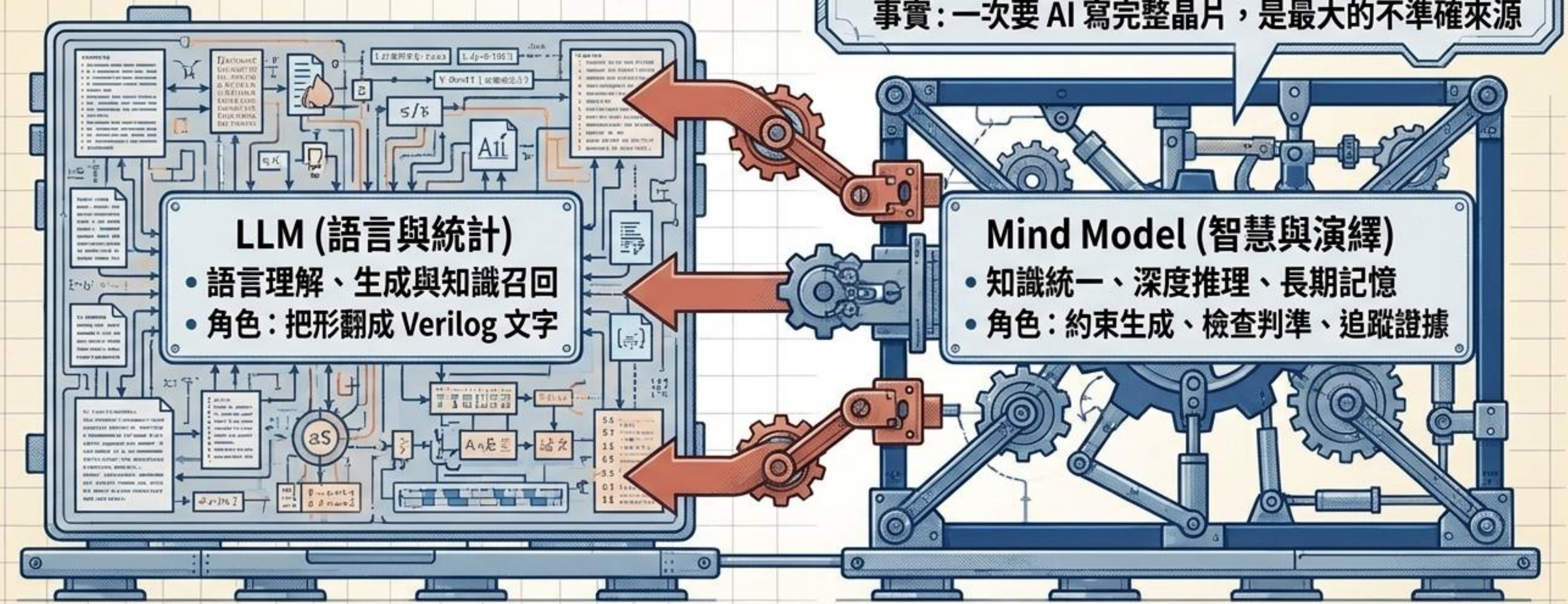


[公理] 設計 = 形 (Type) + 數。先立形、後填數。形錯了，位寬算得再精也毫無意義。

MIND AI

Mind AI 架構：準確性來自底層的分工約束

常見迷思：Prompt 寫得越長越準
事實：一次要 AI 寫完整晶片，是最大的不準確來源



LLM (語言與統計)

- 語言理解、生成與知識召回
- 角色：把形翻成 Verilog 文字

Mind Model (智慧與演繹)

- 知識統一、深度推理、長期記憶
- 角色：約束生成、檢查判準、追蹤證據

模型會換代，但「形」與「判準」留在大腦裡。準確不靠運氣，靠演繹骨架的嚴格約束。

MIND AI

產出形總覽 (The 6 Output Types)

沒有形的流程，產出是散裝的。每一份 RTL 產出，都必須掛在某一口形上；每一口形，都立著一條判準 ∩(T)。

規格形 SpecType [ ]

要做什麼？(屬性為主)

交件判準：每句可度量、可裁決。

介面形 InterfaceType [ ]+( )

怎麼對外？

交件判準：port 對得起來、講得清楚。

模組形 ModuleType { }

由什麼組成？(BOM為主)

交件判準：BOM 判準+remove 分級。

時序形 TimingType ( )

什麼時候動？(容器為主)

交件判準：可示波、可 assertion。

驗證形 VerifyType [ ]{ }( )

怎麼算對？

交件判準：覆蓋達標＋逐條簽收。

模式形 PatternType { }+[ ]

重用什麼？

交件判準：判準隨附＋坑位記錄。

MIND AI

深入解析：規格形 (SpecType) 與介面形 (InterfaceType)

SpecType：規格句的體檢表

鐵律：需求句裡量不出來的形容詞一律退回。

∩(SpecType) 檢查項：

- 可度量：拿什麼數字量它？ ✔ [合規] ✘ [退回]
  (例：115200 bps，8N1 [✔] vs 快速的傳輸 [✘])
- 可歸屬：掛得進哪個括號？ ✔ [合規]
  (例：單一 50 MHz 掛進 ( ) 容器 [✔])
- 可裁決：兩個工程師結論一致？ ☐

InterfaceType：模組的對外契約

鐵律：介面形層級一次講定方向、位寬與時鐘域歸屬。改介面 = 改契約。

UART Port Contract 範例：

訊號	方向/位寬	時鐘域/歸屬
`clk, rst_n`		50 MHz 容器 / 重置域
`tx_data[7:0], tx_start`		傳送面 (50 MHz)
`rx, tx`		序列埠

MIND AI

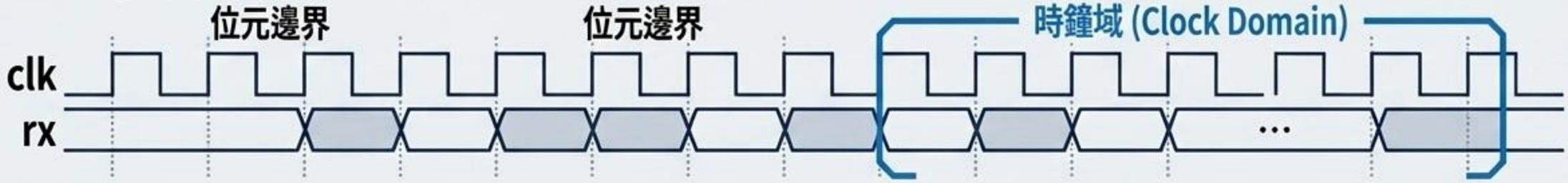
深入解析：模組形 (ModuleType) 與時序形 (TimingType)

ModuleType & remove 三級

用 { } BOM 記組成。移除後同一性喪失？



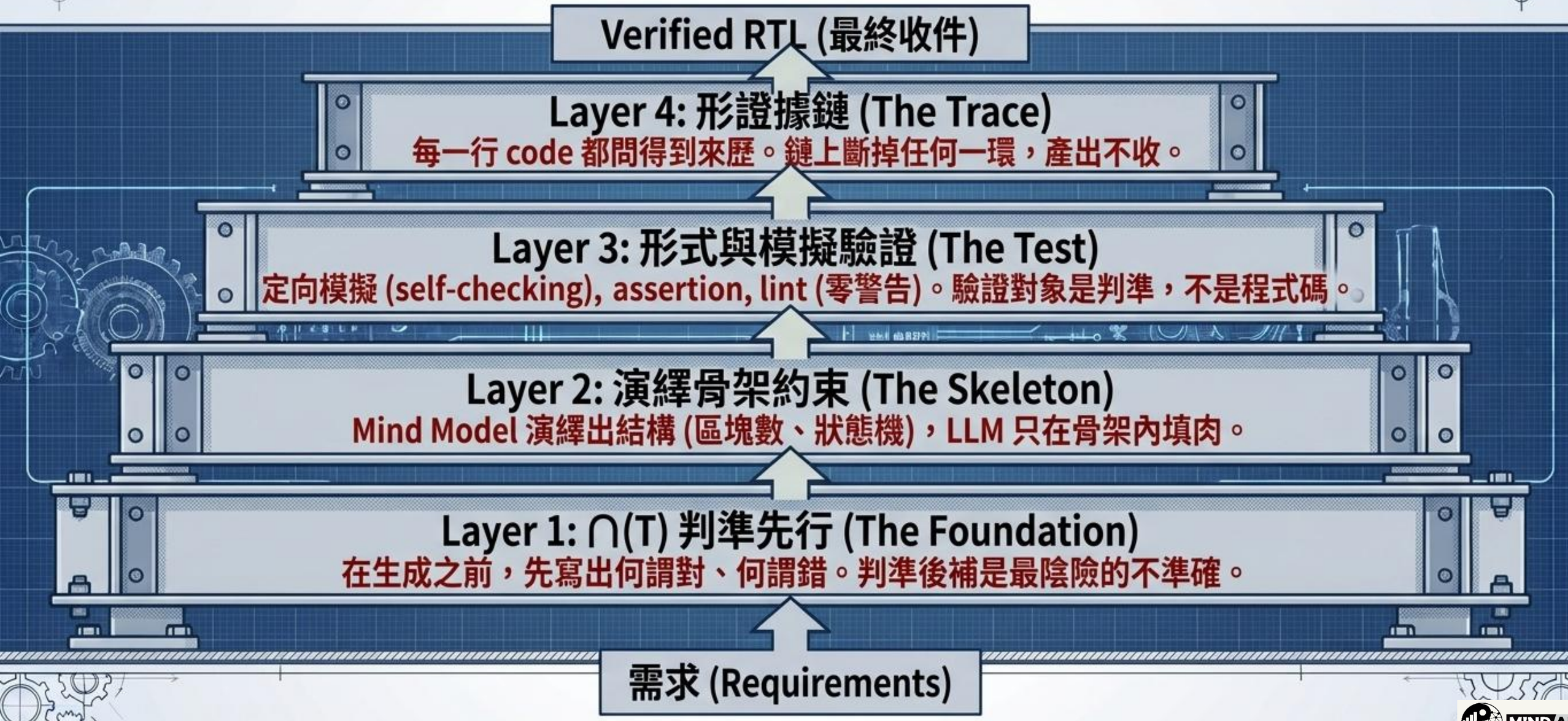
TimingType 的要件 ( ) 容器的動態投影。判準永遠是「可示波的」。



- 時鐘規格 (SDC)
- 幀時序 (baud_tick 對齊)
- 復位序列 (極性/釋放)
- 跨域 (CDC 同步器)

MIND AI

四層準確性保證：信任的結構化來源



MIND AI

形式驗證與形式證據鏈：產出如何驗收

形證據鏈：追溯的藝術

1. 需求：rx 模組的 stop bit 檢查
2. 推理：因果推理 (時間本形招式)
3. 判準：V-3 誤幀率 < 10^{-6} (VerifyType)
4. 證據：演化帳 v1.2 修復紀錄

對了但講不出為什麼對，等於沒有驗證過。

空間 × 時間交叉驗收矩陣

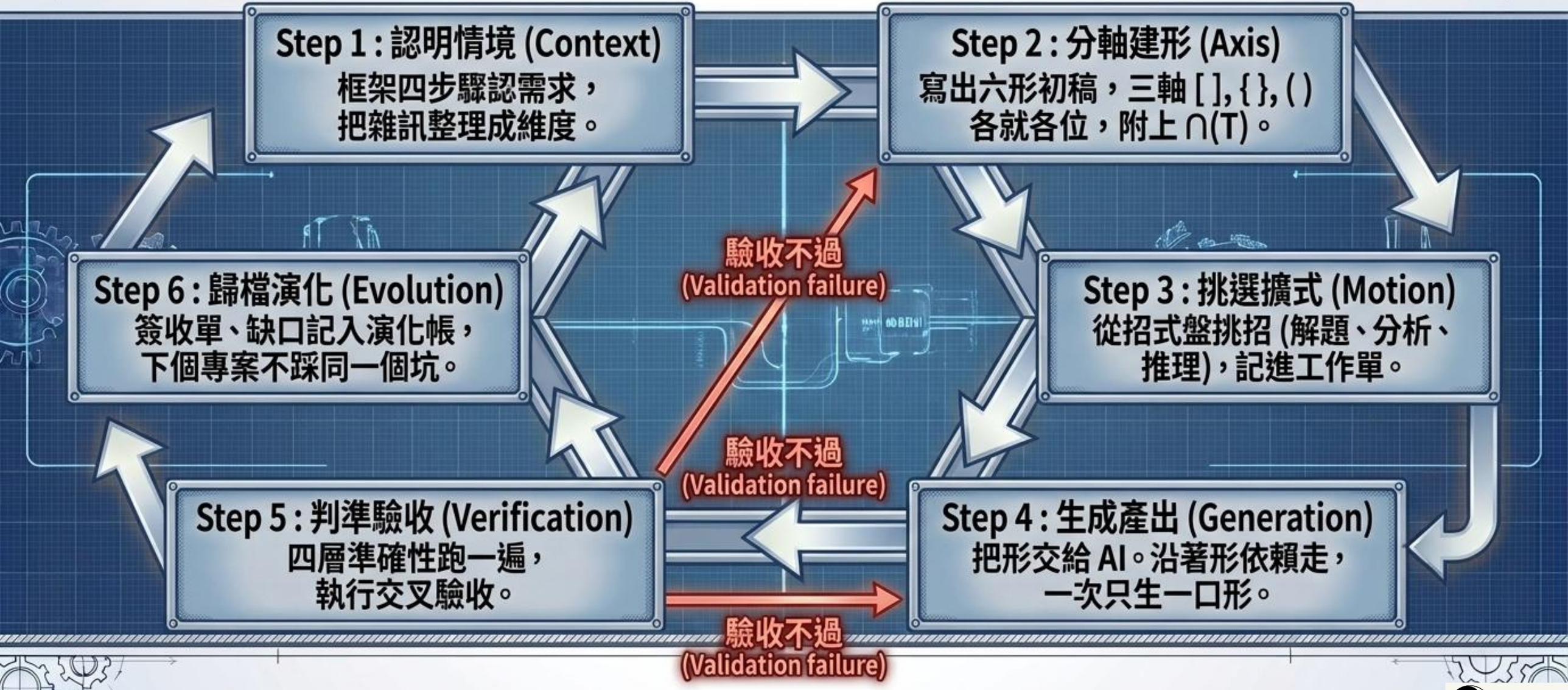
空間 (靜態形)：形完備嗎？
(六形訣、六形齊、判準立、表記全)



時間 (動態招)：招到位嗎？ (推理完整、記錄歸檔)

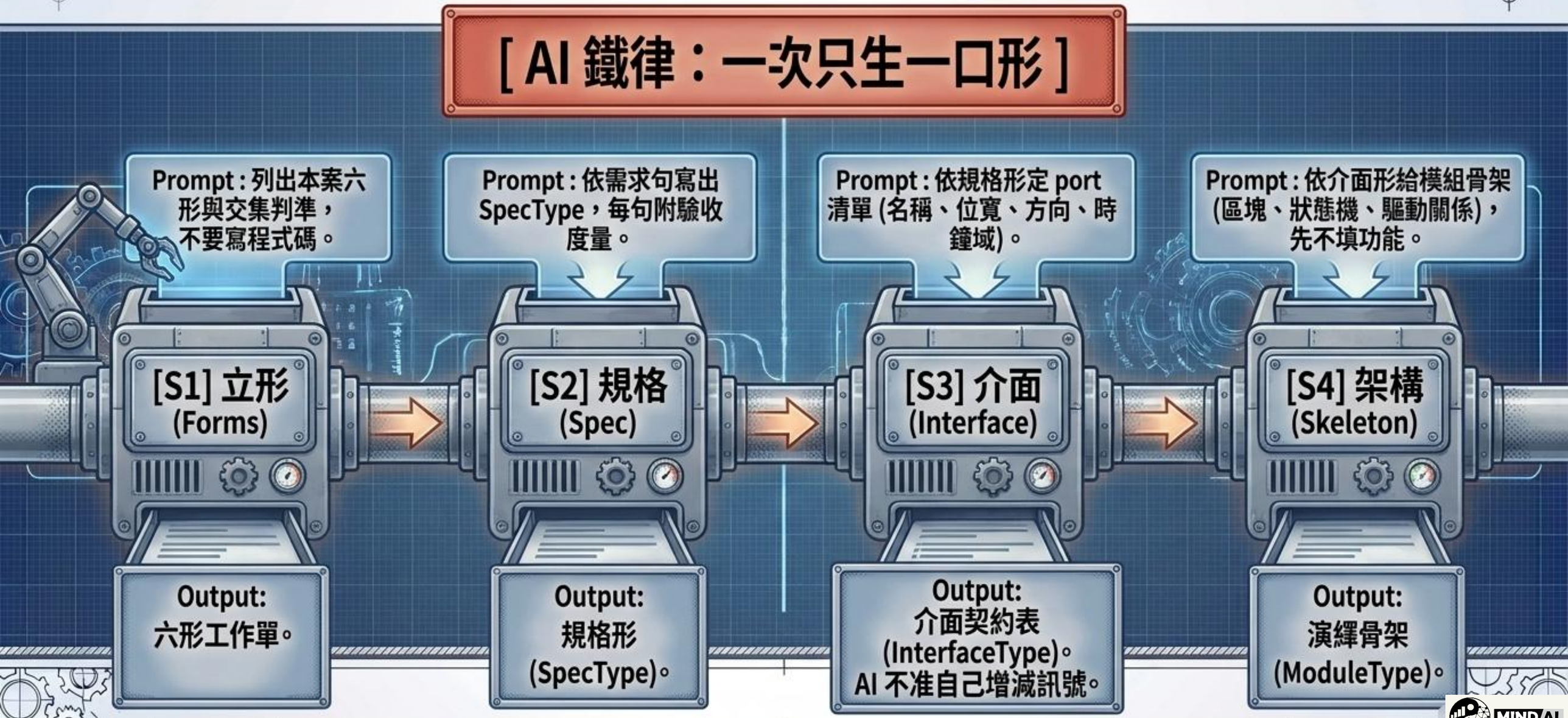
MIND AI

準確使用方法・六步驟 (The 6-Step SOP)



提示詞流水線 S1–S4：從立形到架構 (Prompt Pipeline)

[AI 鐵律：一次只生一口形]



MIND AI

提示詞流水線 S5–S7：從填充到收斂 (Execution & Verification)

[AI 鐵律：一次只生一口形]

[S7] 架斂
(Skeleton)

Prompt：逐區塊實作功能，
一次一個 always 區塊，
保留骨架結構。

Prompt：依 ∩(T) 寫
self-checking testbench
與 assertion。

Prompt：跑覆蓋率、
列出缺口、
整理證據鏈摘要。

[S4] 架構
(Skeleton)

[S5] 填充
(Fill)

[S6] 驗證
(Verify)

[S7] 收斂
(Sign-off)

Output:
六形工作單。

```
uart_tx Snippet:
localparam integer DIV = CLK_FREQ
 / BAUD; // 50e6/115200 = 434
```

```
SystemVerilog Snippet:
!tx_busy |-> tx == 1'b1;
// 閒置時 tx 必為高
```

Output:
最終簽收包
(Verified RTL)。

骨架層鎖死紀律：每個暫存器只有一處驅動。

MIND AI

五種 AI 誤用對照：避開「形變儀式」的陷阱

不準確多半是形的錯，不是語法錯。語法錯 compiler 會抓，形的錯只有準確性保證抓得住。

症狀	修正
形當目錄 vs. 強制判準 症狀：六形列完就收工，判準欄空白。	修正：每口形強制填 ∩(T)，空白不准進流程。
判準後補 vs. 順序鎖死 症狀：RTL 寫完才叫 AI 幫我想測試。	修正：S1 順序鎖死：判準產出必須在程式碼生成之前。
證據斷鏈 vs. 鏈上抽查 症狀：RTL 跑過模擬就交件，回溯不到需求。	修正：抽查證據鏈，斷者不收件。
數先於形 vs. 屬性裁決 症狀：團隊先吵位寬 8 還是 16，再想架構。	修正：先立形後填數；爭議回退到 [ ] 屬性裁決。
介面未鎖 vs. 契約先行 症狀：AI 自己加了沒人要的訊號。	修正：S3 先立契約表，嚴格鎖死不准增減。

MIND AI

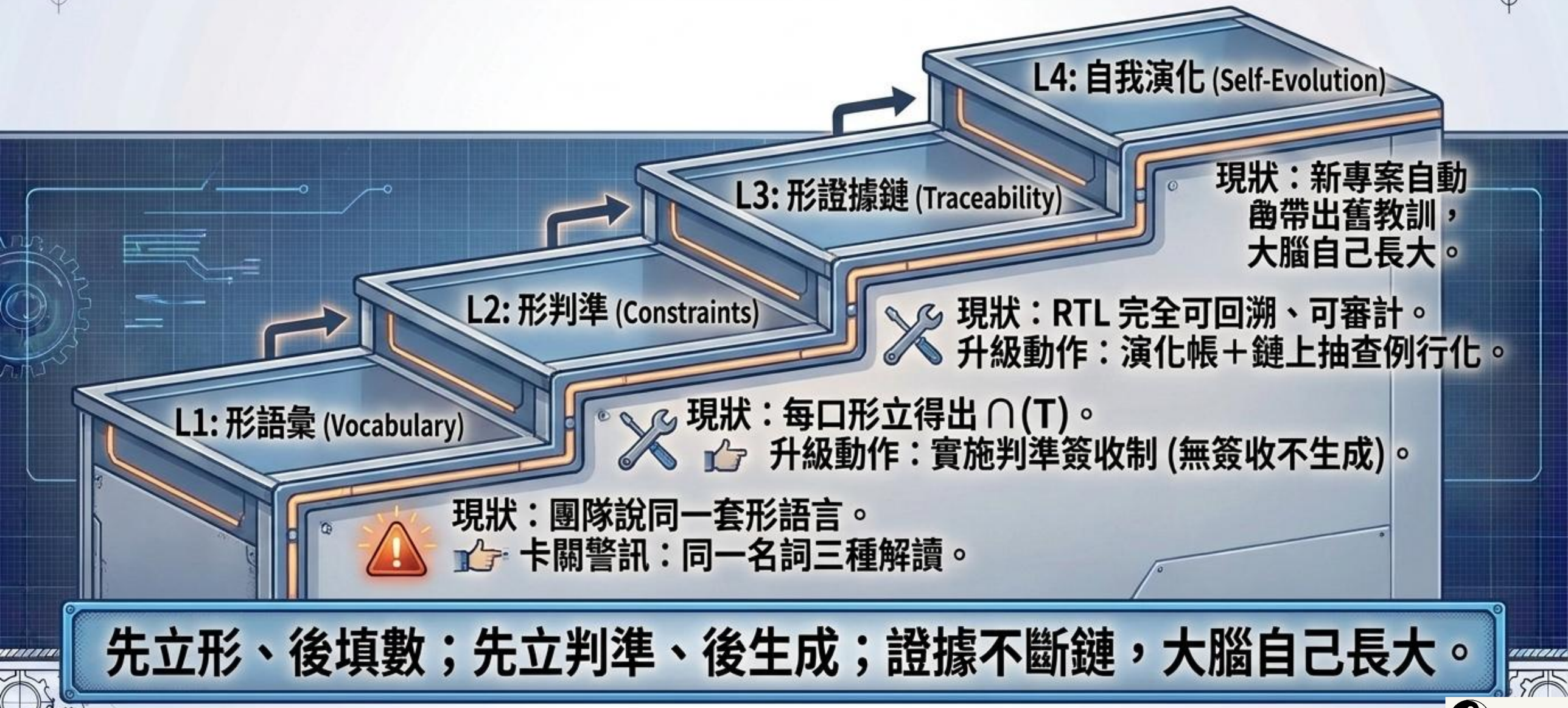
收件前十問：最後的防線
(The Diagnostic Checklist)

警告：任一「否」= 退回重作

- [ ] 每口產出形都立了 ∩(T) 嗎？ → 證據：六形工作單
- [ ] 判準是在生成之前立的嗎？ → 證據：簽收單日期
- [ ] 介面契約表鎖定了嗎？ → 每個 always 區塊只做一件事嗎？
  證據：port 契約表
- [ ] BOM 判準與 remove 分級寫了嗎？ → testbench 是 self-checking 嗎？
  證據：BOM 清單 → 證據：tb 原始碼
- [ ] 跨域訊號都有同步橋嗎？ → 覆蓋率達標、缺口有處置嗎？
  證據：跨域清單 ( ) → 證據：覆蓋率報表
- [ ] 每個 always 區塊只做一件事嗎？ → 招式調用與推理步驟有記錄嗎？
  證據：演繹骨架文件 → 證據：招式工作單
- [ ] 證據鏈回溯得到需求句嗎？
  證據：鏈上隨機抽查

MIND AI

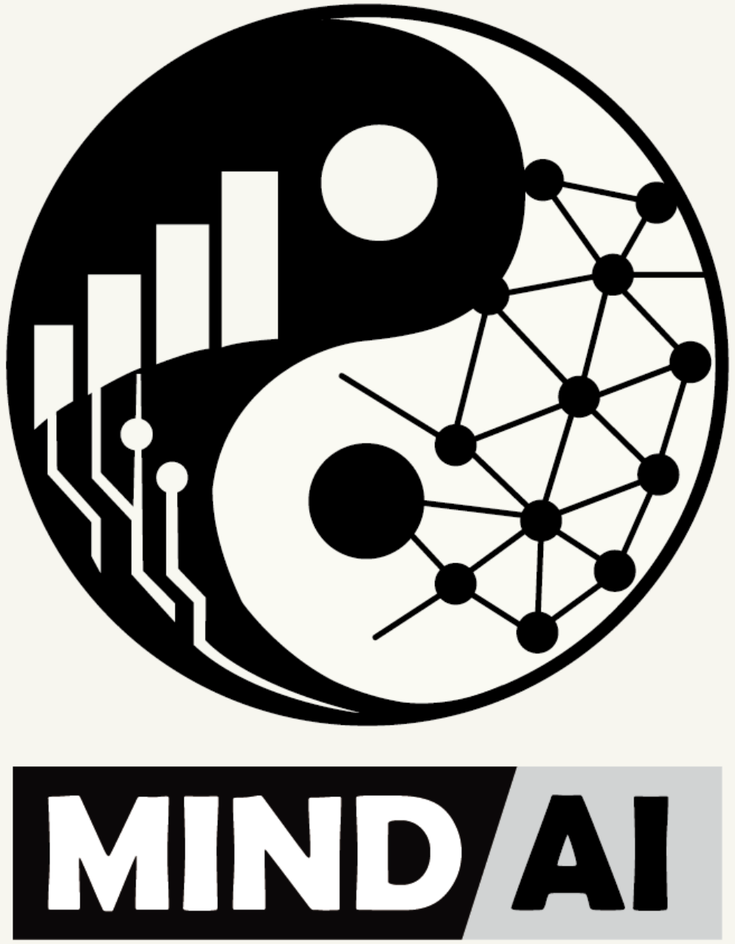
成熟度階梯 (L1-L4)：長出你自己的 IC 設計大腦



先立形、後填數；先立判準、後生成；證據不斷鏈，大腦自己長大。

MIND AI

LonGreat 願景：創造理解世界本質的智慧生命



「讓人工智慧不只是工具，而是真正具備理解、推理、學習與創造能力的智慧生命體系。」

從知識統一，到思維統一，最終邁向智慧統一。
這就是 LonGreat 所追求的 AGI 永恆之道。

www.longreat.net

張長豐
gamma@livemail.tw

張椁嶺
mindaiworks@gmail.com